

Unless state “not coded by me”, the rest of the codes are coded by me.

main.css (start)

```
/* Reset or base styles */
```

```
body {  
    font-family: Arial, sans-serif;  
    margin: 0;  
    padding: 0;  
}
```

```
.container {  
    width: 50%;  
    margin: 0 auto;  
}
```

```
/* Styles specific to author-home.ejs */
```

```
.header.author-home {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    background-color: rgb(210, 11, 117);  
    padding: 10px 20px;  
    text-align: center;  
}
```

```
.header.author-home h1 {  
    margin: 0;  
    color: white;  
}
```

```
.navbar.author-home {  
    display: flex;  
    justify-content: flex-end;  
    margin-top: 10px;  
}
```

```
.navbar.author-home a {
```

```

        color: white;
        text-decoration: none;
        padding: 10px 20px;
        border-radius: 5px;
    }

.navbar.author-home a:hover {
    background-color: lightpink;
    color: black;
}

.footer.author-home {
    margin-top: 50px;
    background-color: #333;
    color: white;
    text-align: center;
    padding: 10px 0;
}

.footer.author-home p {
    margin: 0;
}

.header.author-home .navbar {
    margin-left: auto; /* Push the navbar to the right end */
}

.header.author-home .navbar a[href="/author/author-settings"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

```

```
.header.author-home .navbar a[href="/author/author-settings"]:hover {  
    background-color: lightgray;  
}
```

```
.header.author-home .navbar a[href="/author/login"] {  
    color: black;  
    background-color: white;  
    padding: 10px 20px;  
    border-radius: 5px;  
    text-decoration: none; /* Remove underline */  
}
```

```
.header.author-home .navbar a[href="/author/login"]:hover {  
    background-color: lightgray;  
}
```

```
.header.author-home .navbar a[href="/author/author-edit"] {  
    color: black;  
    background-color: white;  
    padding: 10px 20px;  
    border-radius: 5px;  
    text-decoration: none; /* Remove underline */  
}
```

```
.header.author-home .navbar a[href="/author/author-edit"]:hover {  
    background-color: lightgray;  
}
```

```
.header.author-home .navbar a[href="/"] {  
    color: black;  
    background-color: white;  
    padding: 10px 20px;  
    border-radius: 5px;  
    text-decoration: none; /* Remove underline */  
}
```

```
.header.author-home .navbar a[href="/"]:hover {  
    background-color: lightgray;  
}
```

```
/* Styles specific to reader-home.ejs */
```

```
.header.reader-home {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    background-color: rgb(0, 128, 128);  
    padding: 10px 20px;  
    text-align: center;  
}
```

```
.header.reader-home h1 {  
    margin: 0;  
    color: white;  
}
```

```
.navbar.reader-home {  
    display: flex;  
    justify-content: flex-end;  
    margin-top: 10px;  
}
```

```
.navbar.reader-home a {  
    color: white;  
    text-decoration: none;  
    padding: 10px 20px;  
    border-radius: 5px;  
}
```

```
.navbar.reader-home a:hover {  
    background-color: lightpink;
```

```
        color: black;
    }

.header.reader-home .navbar a[href="/"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

.header.reader-home .navbar a[href="/"]:hover {
    background-color: lightgray;
}

.header.reader-home .navbar a[href="/reader/reader-article"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

.header.reader-home .navbar a[href="/reader/reader-article"]:hover {
    background-color: lightgray;
}

.footer.reader-home {
    margin-top: 50px;
    background-color: #333;
    color: white;
    text-align: center;
    padding: 10px 0;
}
```

```
.footer.reader-home p {
    margin: 0;
}

/* Styles specific to author-edit.ejs */
.header.author-edit{
    display: flex;
    justify-content: space-between;
    align-items: center;
    background-color: rgb(210, 11, 117);
    padding: 10px 20px;
    text-align: center;
}

.header.author-edit h1 {
    margin: 0;
    color: white;
}

.header.author-edit .navbar a[href="/author/author-settings"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

.header.author-edit .navbar a[href="/author/author-settings"]:hover {
    background-color: lightgray;
}

.header.author-edit .navbar a[href="/author/login"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
}
```

```

        border-radius: 5px;
        text-decoration: none; /* Remove underline */
    }

.header.author-edit .navbar a[href="/author/login"]:hover {
    background-color: lightgray;
}

.header.author-edit .navbar a[href="/"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

.header.author-edit .navbar a[href="/"]:hover {
    background-color: lightgray;
}

.header.author-edit .navbar a[href="/author/author-home"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none; /* Remove underline */
}

.header.author-edit .navbar a[href="/author/author-home"]:hover {
    background-color: lightgray;
}

/* Styles specific to reader-article.ejs */
.header.reader-article {
    display: flex;

```

```
    justify-content: space-between;
    align-items: center;
    background-color: rgb(0, 128, 128);
    padding: 10px 20px;
    text-align: center;
}

.header.reader-article h1 {
    margin: 0;
    color: white;
}

.navbar.reader-article {
    display: flex;
    justify-content: flex-end;
    margin-top: 10px;
}

.navbar.reader-article a {
    color: white;
    text-decoration: none;
    padding: 10px 20px;
    border-radius: 5px;
}

.navbar.reader-article a:hover {
    background-color: lightpink;
    color: black;
}

.header.reader-article .navbar a[href="/"] {
    color: black;
    background-color: white;
    padding: 10px 20px;
    border-radius: 5px;
}
```



```
    text-decoration: none; /* Remove underline */
}
```

```
.header.reader-article .navbar a[href="/"]:hover {
    background-color: lightgray;
}
```

```
.footer.reader-article {
    margin-top: 50px;
    background-color: #333;
    color: white;
    text-align: center;
    padding: 10px 0;
}
```

```
.footer.reader-article p {
    margin: 0;
}
```

main.css (end)

author.js (start)

```
const express = require("express");
const router = express.Router();
```

```
/**
 * @desc Displays a page with details for author homepage
 */
router.get("/author-home", (req, res) => {
    res.render("author-home.ejs");
});
```

```
// Route to render author home page
router.get('/author-home', (req, res) => {
    const userName = req.query.username; // Retrieve username from query parameter

    // Render author home page with user name
    res.render('author-home', { userName });
});
```

```
// Route to render the settings page
```

```

router.get('/author-settings', (req, res) => {
  const userName = req.query.username; // Retrieve username from query parameter

  // Query database to fetch user settings based on user_name
  const query = 'SELECT blogtitle FROM users WHERE user_name = ?';
  global.db.get(query, [userName], (err, row) => {
    if (err) {
      console.error(err);
      return res.status(500).send('Error fetching user settings');
    }

    if (!row) {
      return res.status(404).send('User not found');
    }

    const { blogtitle } = row;
    res.render('author-settings', { userName, blogtitle });
  });
});

// Route to handle form submission and update settings
router.post('/author/update-settings', (req, res) => {
  const { userName, blogTitle } = req.body;

  // Update blogtitle in the database
  const query = 'UPDATE users SET blogtitle = ? WHERE user_name = ?';
  global.db.run(query, [blogTitle, userName], function(err) {
    if (err) {
      console.error(err);
      return res.status(500).send('Failed to update blog title');
    }

    console.log(`Blog title updated successfully for ${userName}`);
    res.redirect('/author/author-home'); // Redirect to author home page after update
  });
});

/**
 * @desc Displays a page with details for author login
 */
router.get("/login", (req, res) => {
  res.render("login.ejs");
});

/**
 * @desc Displays a page with details for author edit
 */
router.get("/author-edit", (req, res) => {
  res.render("author-edit.ejs");
});

```

```
});
```

```
// Export the router object so index.js can access it  
module.exports = router;
```

author.js (end)

reader.js (start)

```
const express = require("express");  
const router = express.Router();
```

```
/**  
 * @desc Displays a page with details for reader homepage  
 */  
router.get("/reader-home", (req, res) => {  
  res.render("reader-home.ejs");  
});
```

```
// /**  
// * @desc Displays a page with details for reader articles  
// */  
// router.get("/reader-article", (req, res) => {  
//   res.render("reader-article.ejs");  
// });
```

```
// Route to render reader articles page  
router.get('/reader-article', (req, res) => {  
  // Fetch articles from the database  
  const query = `  
    SELECT articles.articleid, articles.articletitle, articles.articletext,  
articles.datepublished, users.user_name  
    FROM articles  
    INNER JOIN users ON articles.user_id = users.user_id  
    WHERE articles.articlestatus = 'Published'  
    ORDER BY articles.datepublished DESC  
  `;  
  ;
```

```
  global.db.all(query, [], (err, articles) => {  
    if (err) {  
      console.error(err);  
      return res.status(500).send('Error fetching articles');  
    }  
  }
```

```
  // Fetch comments for each article  
  const fetchCommentsPromises = articles.map(article => {  
    return new Promise((resolve, reject) => {  
      const query = `  
        SELECT commenttext  
        FROM comments  
        WHERE articleid = ?  
      `;  
      ;
```

```
      global.db.all(query, [article.articleid], (err, comments) => {  
        if (err) {  
          reject(err);  
        } else {  
          article.comments = comments;  
          resolve();  
        }  
      })  
    });  
  }  
}
```

```

        }
      });
    });
  });

  // Resolve all promises and render the page with fetched data
  Promise.all(fetchCommentsPromises)
    .then(() => {
      res.render('reader-article.ejs', { articles });
    })
    .catch(err => {
      console.error(err);
      res.status(500).send('Error fetching comments');
    });
  });
});

// Export the router object so index.js can access it
module.exports = router;

```

reader.js (end)

users.js (start) - not coded by me

```

/**
 * users.js
 * These are example routes for user management
 * This shows how to correctly structure your routes for the project
 * and the suggested pattern for retrieving data by executing queries
 *
 * NB. it's better NOT to use arrow functions for callbacks with the SQLite
library
*
*/

const express = require("express");
const router = express.Router();

/**
 * @desc Display all the users
 */
router.get("/list-users", (req, res, next) => {
  // Define the query
  query = "SELECT * FROM users"

  // Execute the query and render the page with the results
  global.db.all(query,
    function (err, rows) {
      if (err) {
        next(err); //send the error on to the error handler
      } else {
        res.json(rows); // render page as simple json
      }
    }
  );
});

/**
 * @desc Displays a page with a form for creating a user record
 */

```

```

router.get("/add-user", (req, res) => {
  res.render("add-user.ejs");
});

/**
 * @desc Add a new user to the database based on data from the submitted form
 */
router.post("/add-user", (req, res, next) => {
  // Define the query
  query = "INSERT INTO users (user_name) VALUES( ? );"
  query_parameters = [req.body.user_name]

  // Execute the query and send a confirmation message
  global.db.run(query, query_parameters,
    function (err) {
      if (err) {
        next(err); //send the error on to the error handler
      } else {
        res.send(`New data inserted @ id ${this.lastID}!`);
        next();
      }
    }
  );
});

// Export the router object so index.js can access it
module.exports = router;

```

users.js (end) not coded by me

add-user.ejs (start) - not coded by me

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" type="text/css" href="/main.css" />
  <title>Add User</title>
</head>
<body>
  <h1>Add User</h1>
  <form action="add-user" method="post">
    <p>User: <input id="user" type="text" name="user_name" /></p>
    <button type="submit">Create a new user record</button>
  </form>
</body>
</html>

```

add-user.ejs (end) - not coded by me

author-edit.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Author Edit Page</title>
  <link rel="stylesheet" href="/main.css">

```

```

</head>
<body>
  <header class="header author-edit">
    <div class="container">
      <h1>Article Edit Page</h1>
    </div>
    <nav class="navbar">
      <a href="/" class="btn btn-secondary">Home</a>
      <a href="/author/author-settings" class="btn btn-
secondary">Settings</a>
      <a href="/author/login" class="btn btn-secondary">Login</a>
      <a href="/author/author-home" class="btn btn-secondary">Main</a>
    </nav>
  </header>

  <!-- Rest of your content -->
</body>
</html>

```

author-edit.ejs (end)

author-home.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Author Home Page</title>
  <link rel="stylesheet" href="/main.css">
</head>
<body>
  <header class="header author-home">
    <div class="container">
      <h1>Author Home Page</h1>
    </div>
    <nav class="navbar">
      <a href="/" class="btn btn-secondary">Home</a>
      <a href="/author/author-settings" class="btn btn-
secondary">Settings</a>
      <a href="/author/login" class="btn btn-secondary">Login</a>
      <a href="/author/author-edit" class="btn btn-secondary">Edit
Articles</a>
    </nav>
  </header>

  <!-- Rest of your content -->
</body>
</html>

```

author-home.ejs (end)

author-settings.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Author Settings</title>
  <link rel="stylesheet" href="/main.css">

```

```

</head>
<body>
  <header class="header author-settings">
    <div class="container">
      <h1>Author Settings</h1>
    </div>
    <nav class="navbar">
      <a href="/author-home?username=<%= encodeURIComponent(userName) %>"
class="btn btn-secondary">Back to Author Home</a>
    </nav>
  </header>
  <main class="container">
    <section>
      <h2>Current Settings</h2>
      <form action="/author/update-settings" method="POST">
        <label for="blogTitle">Blog Title:</label>
        <input type="text" id="blogTitle" name="blogTitle" value="<%=
blogtitle %>" required>
        <input type="hidden" name="userName" value="<%= userName %>">
        <br>
        <button type="submit" class="btn btn-primary">Save
Settings</button>
      </form>
    </section>
  </main>
</body>
</html>

```

author-settings.ejs (end)

index.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Blogging Tool</title>
  <!-- CSS files or stylesheets here -->
  <link
href="https://cdn.jsdelivr.net/npm/tailwindcss@2.2.19/dist/tailwind.min.css"
rel="stylesheet">
  <style>
    .bg-turquoise {
      background-color: #40E0D0;
    }
    .hover\:bg-turquoise-dark:hover {
      background-color: #36C2B1;
    }
  </style>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
  <div class="container mx-auto p-4">
    <h1 class="text-3xl font-bold text-center mb-4">Welcome to the Blogging
Tool</h1>
    <div class="flex justify-center">
      <div class="w-full md:w-1/2 bg-white rounded-lg shadow-lg p-8">
        <h2 class="text-xl font-bold mb-4 text-center">Choose Your
Role</h2>
        <div class="flex justify-center space-x-4">

```

```

                <a href="/author/author-home" class="bg-pink-500 hover:bg-
pink-600 text-white font-bold py-4 px-8 rounded-lg text-xl">
                    Author
                </a>
                <a href="/reader/reader-home" class="bg-turquoise hover:bg-
turquoise-dark text-white font-bold py-4 px-8 rounded-lg text-xl">
                    Reader
                </a>
            </div>
        </div>
    </div>
</body>
</html>

```

index.ejs (end)

login.ejs (start)

```

<!-- login.ejs -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <link rel="stylesheet" href="/main.css">
</head>
<body>
    <div class="login-container">
        <h1>Login</h1>
        <form action="/login" method="POST">
            <label for="user_name">Username:</label>
            <input type="text" id="user_name" name="user_name" required>
            <button type="submit" class="btn btn-primary">Login</button>
        </form>
    </div>
</body>
</html>

```

login.ejs (end)

reader-article.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Reader Articles</title>
    <link rel="stylesheet" href="/main.css">
</head>
<body>
    <header class="header reader-articles">
        <div class="container">
            <h1>Reader Articles</h1>
        </div>
        <nav class="navbar">
            <a href="/" class="btn btn-secondary">Home</a>

```



```

        </nav>
    </header>
    <main class="container">
        <section>
            <h2>Latest Articles</h2>
            <% articles.forEach(article => { %>
                <article class="article">
                    <h3><%= article.articletitle %></h3>
                    <p><strong>Published By:</strong> <%= article.user_name %></p>
                    <p><strong>Date Published:</strong> <%= article.datepublished
%></p>

                    <p><%= article.articletext %></p>

                    <% if (article.comments && article.comments.length > 0) { %>
                        <h4>Comments:</h4>
                        <ul>
                            <% article.comments.forEach(comment => { %>
                                <li><%= comment.commenttext %></li>
                                <% }); %>
                            </ul>
                        <% } else { %>
                            <p>No comments yet.</p>
                        <% } %>
                    </article>
                    <hr>
                <% }); %>
            </section>
        </main>
    </body>
</html>

```

reader-article.ejs (end)

reader-home.ejs (start)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Reader Home Page</title>
    <link rel="stylesheet" href="/main.css">
</head>
<body>
    <header class="header reader-home">
        <div class="container">
            <h1>Reader Home Page</h1>
        </div>
        <nav class="navbar">
            <a href="/" class="btn btn-secondary">Home</a>
            <a href="/reader/reader-article" class="btn btn-secondary">Read
Articles</a>
        </nav>
    </header>

    <!-- Rest of your content -->
</body>
</html>

```

reader-home.ejs (end)

db schema.sql (start)

PRAGMA foreign_keys=ON;

BEGIN TRANSACTION;

-- Create users table

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_name TEXT NOT NULL,  
    blogtitle TEXT,  
    blogsubtitle TEXT  
);
```

-- Create email accounts table

```
CREATE TABLE email_accounts (  
    email_account_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    email_address TEXT NOT NULL,  
    user_id INTEGER NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE  
);
```

-- Create articles table

```
CREATE TABLE articles (  
    articleid INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_id INTEGER NOT NULL,  
    datecreated DATE DEFAULT CURRENT_TIMESTAMP,  
    datepublished DATE,  
    datelastedited DATE DEFAULT CURRENT_TIMESTAMP,  
    articletext TEXT,  
    noofviews INTEGER DEFAULT 0,  
    nooflikes INTEGER DEFAULT 0,  
    articletitle TEXT,  
    articlestatus TEXT DEFAULT 'Draft',  
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE  
);
```

-- Create comments table

```
CREATE TABLE comments (  
    commentid INTEGER PRIMARY KEY AUTOINCREMENT,  
    articleid INTEGER NOT NULL,  
    datecommented DATE DEFAULT CURRENT_TIMESTAMP,  
    commenttext TEXT,  
    FOREIGN KEY (articleid) REFERENCES articles(articleid) ON DELETE CASCADE  
);
```

-- Insert default data with blogtitle and blogsubtitle

```
INSERT INTO users (user_name, blogtitle, blogsubtitle) VALUES ('Simon Star',  
'Simonion Blog', 'Exploring the Universe');  
INSERT INTO users (user_name, blogtitle, blogsubtitle) VALUES ('Dianne Dean',  
'Diannean Diary', 'A Journey Through Literature');  
INSERT INTO users (user_name, blogtitle, blogsubtitle) VALUES ('Harry Hilbert',  
'Harryfied Tech Corner', 'The Latest in Technology Trends');
```

```
INSERT INTO email_accounts (email_address, user_id) VALUES ('simon@gmail.com', 1);
```

```
INSERT INTO email_accounts (email_address, user_id) VALUES ('simon@hotmail.com',  
1);
```

```
INSERT INTO email_accounts (email_address, user_id) VALUES ('dianne@yahoo.co.uk',
2);
```

```
INSERT INTO articles (articleid, user_id, datecreated, datepublished,
datelastedited, articletext, noofviews, nooflikes, articletitle, articlestatus)
VALUES (1, 1, DATETIME('now'), NULL, DATETIME('now'), 'Lorem ipsum dolor sit amet,
consectetur adipiscing elit.', 0, 0, 'My Journey into Astronomy', 'Draft');
```

```
INSERT INTO comments (articleid, commenttext) VALUES
(1, 'Great article, very informative!'),
(1, 'I enjoyed reading this.'),
(1, 'Interesting read, keep it up!');
```

```
COMMIT;
```

db_schema.sql (end)

index.js (start)

```
/**
 * index.js
 * This is your main app entry point
 */

// Set up express, bodyparser and EJS
const express = require('express');
const app = express();
const port = 3000;
var bodyParser = require("body-parser");

// Body parser middleware to parse incoming request bodies
app.use(bodyParser.urlencoded({ extended: true }));

// // Session middleware setup
// app.use(session({
//   secret: 'pauliepooh88!!', // a long random string for production
//   resave: false,
//   saveUninitialized: true,
//   cookie: { secure: false } // Set to true if using HTTPS
// }));

// Set view engine to use EJS for rendering
app.set('view engine', 'ejs'); // set the app to use ejs for rendering

// Set view engine to use EJS for rendering
app.use(express.static(__dirname + '/public')); // set location of static files

// Set up SQLite
// Items in the global namespace are accessible throughout the node application
const sqlite3 = require('sqlite3').verbose();
global.db = new sqlite3.Database('./database.db', function(err){
  if(err){
    console.error(err);
    process.exit(1); // bail out we can't connect to the DB
  } else {
    console.log("Database connected");
    global.db.run("PRAGMA foreign_keys=ON"); // tell SQLite to pay attention
    to foreign key constraints
  }
});
```

```

// Handle requests to the home page
app.get('/', (req, res) => {
  res.render('index')
});

// Add Login Routes
app.get('/login', (req, res) => {
  res.render('login'); // Renders your login.ejs file
});

// Login route - POST request to handle form submission
app.post('/login', (req, res) => {
  const { user_name } = req.body;

  // Query the database to check if user_name exists
  global.db.get('SELECT * FROM users WHERE user_name = ?', [user_name], (err,
row) => {
    if (err) {
      console.error(err);
      res.status(500).send('Internal Server Error');
    } else {
      if (row) {
        // User found, redirect to author home page with user_name as
query parameter
        res.redirect(`/author/author-
home?username=${encodeURIComponent(user_name)}`);
      } else {
        // User not found, handle invalid login
        res.status(401).send('Invalid username');
      }
    }
  }
});
});

// Add all the route handlers in usersRoutes to the app under the path /users
const usersRoutes = require('./routes/users');
app.use('/users', usersRoutes);

// Add all the route handlers in authorRoutes to the app under the path /author
const authorRoutes = require('./routes/author');
app.use('/author', authorRoutes);

// Add all the route handlers in readerRoutes to the app under the path /reader
const readerRoutes = require('./routes/reader');
app.use('/reader', readerRoutes);

// Make the web application listen for HTTP requests
app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})

```

index.js (end)